

好的，这是对这份关于“2-3 Ripple-Carry Adder & Addition/Subtraction Logic Unit”PPT内容的详细总结。

2-3 Ripple-Carry Adder & Addition/Subtraction Logic Unit

这份PPT主要介绍了**1-bit 全加器 (Full Adder)**、**n-bit 纹波进位加法器 (Ripple-Carry Adder)**，以及**加法/减法逻辑单元 (Addition/Subtraction Logic Unit)**，并讨论了其门延迟和传播延迟。

1. 1-bit 全加器 (Full Adder - FA)

一个全加器电路有三个输入位和一个两位的输出，输出由一个**和 (Sum)** 和一个**进位输出 (Carry Out)** 组成。

1.1. 输入与输出

- 输入：** X_i (第一个操作数的第 i 位)、 Y_i (第二个操作数的第 i 位)、 C_i (前一位的进位输入)。
- 输出：** S_i (当前位的和)、 C_{i+1} (下一位的进位输出)。

1.2. 逻辑表达式

- 和 (S_i) 的表达式：**

$$S_i = \bar{x}_i \bar{y}_i c_i + \bar{x}_i y_i \bar{c}_i + x_i \bar{y}_i \bar{c}_i + x_i y_i c_i = x_i \oplus y_i \oplus c_i$$

- 其逻辑图由一个三输入的异或门 (XOR) 实现。

- 进位输出 (C_{i+1}) 的表达式：**

$$C_{i+1} = y_i c_i + x_i c_i + x_i y_i$$

- 其逻辑图由三个 AND 门和一个 OR 门实现。

2. n-bit 纹波进位加法器 (Ripple-Carry Adder)

n-bit 纹波进位加法器由 n 个全加器 (FA) 级联组成。

2.1. 4-bit 加法器示例

- 一个 4-bit 加法器由**四个全加器**组成。
- 总输入 (9个):**
 - 两个 4-bit 数: $A_3 A_2 A_1 A_0$ 和 $B_3 B_2 B_1 B_0$ 。
 - 一个初始进位输入 C_I 。
- 总输出 (5个):**
 - 一个 4-bit 和: $S_3 S_2 S_1 S_0$ 。
 - 一个进位输出 C_O 。
- 结构：** 每一级全加器将前一级的进位输出 (C_i) 作为其进位输入 (C_{in})，进位信号逐级“纹波”传递。

2.2. n-bit 组织

- 输入：** $X = x_{n-1} \dots x_1 x_0$, $Y = y_{n-1} \dots y_1 y_0$, c_0 。
- 输出：** $S = s_{n-1} \dots s_1 s_0$, c_n 。
- 最右侧的 FA (#0) 是**最低有效位 (LSB) 位置**，接收 c_0 。
- 最左侧的 FA (#n-1) 是**最高有效位 (MSB) 位置**，输出 s_{n-1} 和 c_n 。

2.3. 溢出检测 (Overflow Detect)

溢出 (*Overflow*) 可以用以下逻辑表达式表示:

$$\text{Overflow} = x_{n-1} y_{n-1} \bar{S}_{n-1} + \bar{x}_{n-1} \bar{y}_{n-1} s_{n-1}$$

3. 门延迟和传播延迟

3.1. 门延迟 (Gate Delay)

- **定义：** 从输入出现到输出上出现正确结果之间的一小段时间被称为**门延迟**。
- **假设：** 对于该课程，假设所有门的延迟都是一个相同的、小的常数。

3.2. 纹波进位加法器的传播延迟

- **最长路径：** 延迟取决于逻辑门网络中最长信号传播路径上的门延迟数。对于 n -bit 纹波进位加法器，最长路径是从 LSB 位置的输入 x_0, y_0, c_0 到 MSB 位置的输出 c_n 和 s_{n-1} 。
- **假设：** 假设 AND、OR 或 XOR 门的门延迟为 T 。
- **延迟计算 (基于 c_0 到 c_n 的路径)：**
 - 进位输出 c_{i+1} (由 AND-OR 逻辑实现) 相对于进位输入 c_i 的延迟是 $2T$ 。
 - 进位 c_n 相对于 c_0 的延迟通过 n 级传播：

$$\text{Delay}(c_n) = 2(n-1)T + 2T = (2n)T$$

- 和 s_{n-1} 相对于 c_0 的延迟：

$$\text{Delay}(s_{n-1}) = 2(n-1)T + T = (2n-1)T$$

4. 加法/减法逻辑单元 (Addition/Subtraction Logic Unit)

该单元通过一个**加/减控制**信号来执行加法或减法。

- **执行加法：**
 - **Add/Sub Control = 0**
 - $c_0 = 0$
 - $Y'_i = Y_i \oplus 0 = Y_i$ (输入 Y 保持不变)
- **执行减法：**
 - **Add/Sub Control = 1**
 - $c_0 = 1$
 - $Y'_i = Y_i \oplus 1 = \overline{Y_i}$ (输入 Y 取反，加上 $c_0 = 1$ 构成 Y 的二补数)

传播延迟： 考虑到输入 Y 上的 XOR 门延迟，所有和位 (s_{n-1} 到 s_0) 在 $2n$ 个门延迟后即可获得。

5. 课堂练习 (Quiz)

1. 在 1-bit 全加器中， s_i 的表达式是：**D. $x_i \oplus y_i \oplus c_i$**
2. 在 1-bit 全加器中， c_{i+1} 的表达式是：**C. $x_i y_i + x_i c_i + y_i c_i$**
3. 如果要构建一个 64-bit 加法器，需要多少个 4-bit 纹波进位加法器？**B. 16** ($64/4 = 16$)

好的，这是一份关于“快速加法器设计”的PPT内容总结，已用Markdown格式呈现，并涵盖了所有要点：

2-4 快速加法器设计 PPT 内容总结

这份PPT主要讨论了** n 位纹波进位加法器 (Ripple-Carry Adder)的缺点，并介绍了通过超前进位加法 (Carry-Lookahead Addition, CLA)**来解决延迟问题的方法和实现细节。

1. 纹波进位加法器 (Ripple-Carry Adder) 的缺点

纹波进位加法器由串联的全加器 (FA) 组成，其主要缺点在于产生输出时延过大。

- **时延分析 (假设一个门延迟为 T)：**
 - 最高位和 S_{n-1} 的时延： $(2n-1)T$ 。

- 最终进位 C_n 的时延: $(2n)T$ 。
- 解决方案:
 - i. 在实现纹波进位逻辑设计时, 使用最快的电子技术。
 - ii. 使用**增强的逻辑门网络结构** (即超前进位加法)。

2. 超前进位加法 (Carry-Lookahead Addition)

超前进位加法通过并行计算进位, 显著减少了时延。

核心函数：进位产生和进位传播

对于任何位级 i , 我们可以定义以下函数来重写进位 c_{i+1} 的表达式:

- 进位产生函数 (G_i):** $G_i = x_i y_i$ (如果 x_i 和 y_i 都是 1, 则本级产生一个进位)。
- 进位传播函数 (P_i):** $P_i = x_i \oplus y_i$ 或 $P_i = x_i + y_i$ 。
 - 注: PPT中给出两种定义, 但电路图和扩展逻辑函数使用 $P_i = x_i \oplus y_i$ 。
- 进位 c_{i+1} 的递归表达式:** $c_{i+1} = G_i + P_i c_i$ 。
- 和 S_i 的表达式:** $S_i = x_i \oplus y_i \oplus c_i$ 。

逻辑函数扩展

将 c_i 迭代代入进位表达式 $c_{i+1} = G_i + P_i c_i$, 可以得到 c_{i+1} 完全由输入 X, Y 和初始进位 c_0 决定的表达式。

$$c_{i+1} = G_i + P_i G_{i-1} + P_i P_{i-1} G_{i-2} + \dots + P_i P_{i-1} \dots P_1 G_0 + P_i P_{i-1} \dots P_0 c_0$$

n位CLA的时延

- 所有**进位** c_i 可以在输入信号 X, Y 和 c_0 施加后经过 **3个门延迟** 获得。
- 所有**和位** S_i 可以在输入信号 X, Y 和 c_0 施加后经过 **4个门延迟** 获得。

3. 4位超前进位加法器 (4-bit CLA)

4位超前进位加法器是CLA的基本单元, 它由4个“B cell”位级单元和一个“超前进位逻辑”块组成。

进位逻辑函数

基于 $c_{i+1} = G_i + P_i c_i$ 和 $i = 0, 1, 2, 3$, 可以得到 c_1, c_2, c_3, c_4 的完整表达式:

- $c_1 = G_0 + P_0 c_0$
- $C_2 = G_1 + P_1 G_0 + P_1 P_0 c_0$
- $C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 c_0$
- $C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 c_0$

时延对比 (n=4)

加法器类型	输出	时延 (以门延迟 T 计)
4位 CLA	c_4 (进位)	$3T$
	S_3 (和)	$4T$
4位 纹波加法器	c_4 (进位)	$8T$
	S_3 (和)	$7T$

4. 构建更长的超前进位加法器

4位CLA的设计**不能直接扩展**到更长的操作数尺寸。

限制因素：扇入 (Fan-In)

- 扇入 (Fan-in)：逻辑门的输入数量。
- 扇出 (Fan-out)：逻辑门输出所驱动的门输入数量。
- 实际电路不容许过大的扇入和扇出，因为它们会对传播延迟和电路速度产生不利影响。
- 问题示例：在生成 c_{i+1} 的表达式中，最后一个 AND 门 和 OR 门 的扇入是 $i + 2$ 。随着 i 增大，扇入迅速增加，超过了实际电路的限制。

解决方案：用4位CLA级联构建32位加法器

- 可以通过级联 8个4位超前进位加法器 来构建一个32位加法器。
- 时延分析 (32位级联CLA)：
 - $c_4, c_8, c_{12}, \dots$ 的产生依赖于前一个块的输出进位。
 - 最终进位 c_{32} 的时延约为 $17T$ 。
 - 最高和位 S_{31} 的时延约为 $18T$ 。
- 时延对比 (32位)：
 - 32位级联CLA: $S_{31} \approx 18T, C_{32} \approx 17T$ 。
 - 32位纹波加法器: $S_{31} = (2 \times 32 - 1)T = 63T, C_{32} = 2 \times 32T = 64T$ 。

5. 测验要点回顾

- 16位纹波加法器 计算 S_{13} 的门延迟: $2(13)T + T = 27T$ (对应选项 C)。
- 超前进位加法器的优点是：加速进位生成 (对应选项 D)。
- 进位传播函数 P_i 的表达式是: $x_i \oplus y_i$ 或 $x_i + y_i$ (对应选项 A/B, 取决于具体定义, 但选项中给出 $x_i + y_i$ 和 $x_i \oplus y_i$)。
- 进位产生函数 G_i 的表达式是: $x_i y_i$ (对应选项 C)。
- 在4位CLA中, 所有进位可以在 3个门延迟 后获得 (True)。

好的，这是一份关于“无符号数乘法”的PPT内容总结，已用Markdown格式呈现，详实易懂，并涵盖了所有要点。

 2-5 无符号数乘法 (Multiplication of Unsigned Numbers) PPT 内容总结

这份PPT介绍了无符号二进制数乘法的三种主要方法：手动算法、阵列乘法器 (Array Multiplication) 和时序乘法器 (Sequential Multiplication)。

1. 手动乘法算法 (Manual Multiplication Algorithm)

手动乘法模拟了我们在十进制中使用的传统长乘法，但应用于二进制数。

- 基本原理：
 - 乘法是将被乘数 (M) 乘以乘数 (Q) 的每一位。
 - 如果乘数位为 '1', 则将被乘数按相应位置移位后加入部分积。
 - 如果乘数位为 '0', 则加入 '0'。
 - 将所有的部分积 (Partial Products) 相加，得到最终的乘积 (Product)。
- 结果位数：两个 n 位二进制整数相乘，其乘积最多可达 $2n$ 位。
 - 例如: $M = 1101$ (4位), $Q = 1011$ (4位) $\rightarrow$ 乘积 $P = 10001111$ (8位)。

2. 阵列乘法器 (Array Multiplication)

阵列乘法器是一种并行硬件实现，用于快速计算乘积。

硬件结构

阵列乘法器使用一个二维网格结构，由多个相同的单元组成。

- 基本单元：每个单元包含一个 AND 门 和一个 全加器 (FA)。
 - AND 门计算部分积的单个位 $m_j q_i$ 。
 - 全加器将这个部分积位、传入的部分积位和传入的进位进行相加。

- 工作方式：**所有部分积 m_iq_j 同时产生，并通过阵列中的全加器进行并行相加，避免了时序乘法中的重复移位和相加操作。

硬件成本 (对于 $n \times n$ 阵列乘法器)

- AND 门** 数量: n^2 。
- 全加器 (FA)** 数量: $n(n-1)$ 。
 - 例如: 对于 $n = 32$ 位乘法器, 需要 1024 个 AND 门和 992 个 FA。

3. 时序乘法器 (Sequential Multiplication)

时序乘法器使用一个**单个 n 位加法器**在多个时钟周期内完成乘法操作, 节省了硬件资源。

硬件组件

时序电路的核心是一个 n 位加法器, 以及用于存储和控制的寄存器和逻辑。

- 寄存器 M:** 存储被乘数 (Multiplicand)。
- 寄存器 Q:** 存储乘数 (Multiplier)。
- 寄存器 A:** 最初为 0, 用于累加部分积。
- 寄存器 C:** 1位进位寄存器 (Carry)。
- n 位加法器:** 用于将 A 加上 M (或 0)。
- 多路复用器 (MUX):** 根据乘数 Q 的最低位 Q_0 选择 M 或 0 送入加法器。
- 控制序列器 (Control Sequencer):** 根据 Q_0 信号和计数器 (Count) 控制操作流程。

算法流程 (Flowchart)

该过程重复 n 次 (其中 n 是操作数的位数)。

- 初始化:** $C, A = 0$; $M =$ 被乘数; $Q =$ 乘数; $Count = n$ 。
- 判断 Q_0 :** 检查乘数 Q 的最低位 (Q_0)。
 - 如果 $Q_0 = 1$ (Yes): 执行**加法**操作 $\rightarrow C, A = A + M$ 。
 - 如果 $Q_0 = 0$ (No): 执行**不加**操作 $\rightarrow$ 保持 A 不变。
- 移位:** 将 C, A, Q 寄存器作为一个整体向**右移位**一位。
- 循环:** $Count = Count - 1$ 。如果 $Count \neq 0$, 则重复步骤 2。
- 结束:** 当 $Count = 0$ 时, 乘积存储在 A 和 Q 寄存器中。

好的, 让我们来演示一个时序乘法器的计算过程。

时序乘法器示例: $A \times B$

我们将使用您在测验中给出的例子进行演示。

- 被乘数 (Multiplicand)** $M = A = 00101$
- 乘数 (Multiplier)** $Q = B = 10101$
- 位数** $n = 5$

寄存器设置:

- C** (1位进位寄存器)
- A** (n 位部分积累加寄存器)
- Q** (n 位乘数寄存器)
- M** (n 位被乘数寄存器)

周期 (Cycle)	操作 (Action)	C	A	Q	说明 (Q ₀)
初始配置	初始化	0	00000	10101	$M = 00101$
1	$Q_0 = 1 \rightarrow$ 加法 $C, A = A + M$	0	00101	10101	$00000 + 00101 = 00101$
	右移 C, A, Q	0	00010	11010	Q 的最低位 '1' 移入 A

周期 (Cycle)	操作 (Action)	C	A	Q	说明 (Q ₀)
2	$Q_0 = 0 \rightarrow$ 不加	0	00010	11010	A 保持不变
	右移 C, A, Q	0	00001	01101	Q 的最低位 '0' 移入 A
3	$Q_0 = 1 \rightarrow$ 加法 $C, A = A + M$	0	00110	01101	$00001 + 00101 = 00110$
	右移 C, A, Q	0	00011	00110	Q 的最低位 '1' 移入 A
4	$Q_0 = 0 \rightarrow$ 不加	0	00011	00110	A 保持不变
	右移 C, A, Q	0	00001	10011	Q 的最低位 '0' 移入 A
5	$Q_0 = 1 \rightarrow$ 加法 $C, A = A + M$	0	00110	10011	$00001 + 00101 = 00110$
	右移 C, A, Q	0	00011	01001	Q 的最低位 '1' 移入 A
结果	乘积 P		00011	01001	

最终乘积 P (Product) 是 A 和 Q 寄存器的内容拼接起来：0001101001。

验证：

- $A = 00101_2 = 5_{10}$
- $B = 10101_2 = 21_{10}$
- $A \times B = 5 \times 21 = 105_{10}$
- $P = 0001101001_2 = 32 + 16 + 4 + 1 = 105_{10}$ (结果正确)

您对时序乘法的流程是否还有疑问？或者您想了解阵列乘法器是如何实现并行计算的？
好的，这个PPT的内容总结如下，主要围绕有符号数乘法和Booth算法展开：

2-6 有符号数乘法总结

一、Booth 算法引入及必要性

- 问题所在： 对有负数的乘法，直接使用传统的无符号数乘法将无法得到正确的结果。
 - 例子： 如果将 $M = 1001$ 和 $Q = 0011$ 解释为有符号的 2's 补码数，则 $M = -7$, $Q = 3$ 。直观乘法结果 $P = 00011011$ 解释为 2's 补码是 27 (即 $(-7) \times 3 \neq 27$)。
- 传统乘法的缺陷： 直接使用传统乘法对有符号数（例如 $M = 1001, Q = 0011$ ）进行计算，结果 11101011 (2's 补码为 -21) 也不正确。
- 解决方案：
 - 转换方法： 转换为无符号数相乘，然后将结果再转换回有符号数。这种方法可行但慢且麻烦。
 - Booth 算法： 可以直接处理有符号数（2's 补码）的乘法。

二、Booth 算法的核心原理

1. 减少部分积的数量

- Booth 算法通过对乘数的位进行重新编码 (Recoding) 来减少部分积的数量。
- 连续的 0 组： 不生成新的部分积，只需将部分积右移一位（每遇到一个 0）。
- 连续的 1 组： 相比于 m 个 1 产生 m 个部分积，它会生成少于 m 个部分积。
 - 例子： $0011110 = 0100000 - 0000010$ (十进制： $30 = 32 - 2$)。这用两次操作 (一个 +1 和一个 -1) 代替了四个 +1 操作。

2. 乘数位编码规则 (Recoding)

Booth 算法通过检查乘数位 Q_i 和它右侧的位 Q_{i-1} 来确定操作 (选择的乘数版本 M 的系数):

Q_i	Q_{i-1}	操作	选定的乘数版本
0	0	$0 \times M$	$0 \times M$
0	1	$+1 \times M$	$+1 \times M$

Q_i	Q_{i-1}	操作	选定的乘数版本
1	0	$-1 \times M$	$-1 \times M$
1	1	$0 \times M$	$0 \times M$
: Booth 算法乘数编码表			

- **例子：** 乘数 001011001110101100 的 Booth 编码为 $-0 + 1 - 1 + 1\ 0 - 1\ 0 + 1\ 0\ 0 - 1 + 1 - 1 + 1\ 0 - 1\ 0\ 0$ 。

三、Booth 算法的硬件组织与流程

1. 硬件组织

- **M 寄存器：** 存放被乘数 (Multiplicand)。
- **Q 寄存器：** 存放乘数 (Multiplier)。
- **Q_{-1} 寄存器：** 一个 1-bit 寄存器，逻辑上位于 Q 寄存器的最右边 (Q_0)。
- **A 寄存器：** 初始为 0，用于累加部分积。
- **控制逻辑：** 每次扫描乘数的 Q_0 位及其右侧的 Q_{-1} 位。
- **结果：** 乘法结果最终将出现在 A 寄存器和 Q 寄存器中。

2. 运算流程 (Flowchart)

- **初始化：** $A = 0, Q_{-1} = 0, M =$ 被乘数, $Q =$ 乘数, Count= n (位数)。
- **循环 (Count $\neq$ 0):**
 - 检查 Q_0, Q_{-1} 。
 - 操作：**
 - 00 或 11: 无加减操作 (即 $A = A + 0 \times M$)。
 - 10: $A = A - M$ (即 $A = A + (-1 \times M)$)。
 - 01: $A = A + M$ (即 $A = A + (+1 \times M)$)。
 - 算术右移 (Arithmetic Shift Right)：** 将 A, Q, Q_{-1} 寄存器内容整体算术右移一位。
 - 算术右移规则：** A 的最左位 A_{n-1} 不仅移入 A_{n-2} ，还保持在 A_{n-1} 。
 - 计数：** Count = Count - 1。
- **结束：** 当 Count = 0 时，运算结束。

四、Booth 算法的优缺点

1. 优点 (Advantages)

- **统一处理：** 统一处理正数和负数的乘法。
- **效率提升：** 当乘数包含多个大块的连续 1 时，所需加法次数减少，从而提高效率。

2. 性能 (Performance)

- **平均速度：** Booth 算法的乘法速度**平均而言**与普通算法相同。
- **非总是更快：** 它**并非总是**比普通算法快 (例如乘数是交替的 010101... 时为最差情况)。

好的，这是一份关于“整数除法 (Integer Division)”的幻灯片内容的详细总结，涵盖了要点和示例。

2-7 整数除法总结 (Integer Division)

这份幻灯片主要介绍了计算机组织与架构中的整数除法，特别是针对**正整数的手动除法**、**恢复除法 (Restoring Division)** 和 **非恢复除法 (Non-Restoring Division)** 算法。

1. 正整数的手动除法 (Manual Division of Positive Integers)

手动除法通过一个二进制的例子进行了演示，其过程类似于我们熟悉的十进制长除法。

示例: 计算 $100010010 \div 1101$

- 除数 (Divisor): 1101
- 被除数 (Dividend): 100010010
- 商 (Quotient): 000010101
- 余数 (Remainder): 1
- 部分余数 (Partial Remainders): 除法过程中的中间余数。

2. 恢复除法 (Restoring Division)

恢复除法是一种在硬件中实现二进制除法的算法。

硬件结构 (Hardware)

硬件结构包括:

- A 寄存器:** 存放被除数的高位, 最终存放余数。
- Q 寄存器:** 存放被除数的低位, 最终存放商。
- M 寄存器:** 存放除数 (Divisor)。
- n+1 位加法/减法器 (n+1-bit adder):** 用于执行 A-M 或 A+M 操作。
- 控制序列器 (Control Sequencer):** 控制整个流程。
- 操作:** 主要包括左移 (Shift Left) A, Q 寄存器、减法 (A - M)、商位设置。

流程图 (Flowchart)

该算法的核心步骤在 n 次循环中重复执行:

- 初始化: $A = 0$, M = 除数, Q = 被除数, $Count = n$ 。
- 循环:
 - 左移 A, Q 。
 - 计算 $A = A - M$ 。
 - 判断 A 的符号:
 - 如果 $A < 0$ (减法失败): $Q_0 = 0$, 并执行 恢复 (Restore) 操作 $A = A + M$ 。
 - 如果 $A \geq 0$ (减法成功): $Q_0 = 1$ 。
 - $Count = Count - 1$ 。
- 结束: 当 $Count = 0$ 时循环结束, 商在 Q 中, 余数在 A 中。

运算示例 (Example)

幻灯片提供了一个详细的恢复除法运算示例, 展示了各个周期中 A 和 Q 寄存器的变化。

3. 非恢复除法 (Non-Restoring Division)

非恢复除法是对恢复除法的一种改进, 旨在避免在减法失败后执行恢复操作 ($A = A + M$)。

算法改进 (Algorithm Improvement)

考虑前一个算法中减法操作之后的序列:

- 如果 A 为正 ($A \geq 0$): 下一个操作是“左移并减去 M ”, 相当于 $2A - M$ 。
- 如果 A 为负 ($A < 0$):
 - 恢复除法中, 操作是 $A + M$, 然后“左移并减去 M ”, 即 $2(A + M) - M = 2A + 2M - M = 2A + M$ 。
 - 非恢复除法直接利用这个等价关系: 如果 A 为负, 直接执行 $2A + M$ (即左移 A, Q , 然后执行 $A = A + M$), 而跳过恢复步骤。

运算示例 (Example)

幻灯片提供了一个非恢复除法运算的示例和最终的测验示例。

在非恢复除法中，余数的**最终恢复 (Restoring Remainder)** 可能在所有循环结束后进行。如果最后的余数 A 是负数，则需要通过 $A = A + M$ 将其转换为正确的正余数。

4. 结论与注意事项 (Conclusion and Notes)

- 对于**有符号操作数 (signed operands)** 的除法，没有类似于有符号乘法的简单直接算法。
- 在除法中，通常将操作数**转换成正值**进行处理。
- 使用恢复除法或非恢复除法等算法完成后，再将结果**转换回正确的有符号值** (如果需要)。

好的，这是一份关于“浮点数表示 (Floating-point Representation) 与 IEEE 754 标准”幻灯片内容的详细总结，涵盖了要点和示例。

2-8 浮点数表示与 IEEE 754 标准总结

这份幻灯片介绍了计算机中表示数字的两种主要格式：定点数和浮点数，并详细讲解了业界广泛采用的 **IEEE 754 浮点数标准**。

1. 数字格式 (Number Formats)

根据二进制小数点的位置是否固定，有两种数字格式：

- 定点数 (Fixed-point numbers)**：二进制小数点的位置是固定的。例如，整数的小数点隐含在其最右端。
 - 表示类型**：无符号整数、有符号整数、有符号小数（纯分数）。
 - 局限性**：无法表示**非常大的整数**，也无法表示**非常小的分数**。
 - 范围示例** (32位有符号定点数)：
 - 解释为整数： $[-2^{31}, 2^{31} - 1]$ 。
 - 解释为分数： $[-1, 1 - 2^{-(31)}]$ 。
- 浮点数 (Floating-point numbers)**：小数点的位置是可变的，并会随着计算自动调整。其位置必须在表示中**显式给出**。

2. 浮点数表示 (Floating-point Representation)

浮点数类似于科学记数法，其编码形式包含三个主要部分：

$$\text{数值形式} = (-1)^S \times M \times 2^E$$

组成部分	含义	备注
符号位 (S)	决定数值的正负 (0为正, 1为负)	
尾数 (M)	包含有效数字的分数部分	通常采用原码或补码表示
指数 (E)	编码后的指数部分	实际指数是 e

偏差/移码表示 (Excess or Biased Notation)

为避免使用补码表示负指数，浮点数通常采用**移码或偏差 (Bias)** 表示指数 E 。

- 一个固定的值（偏差）从指数域 E 中减去，得到**真实指数 e** 。
- 计算公式**： $E = e + (2^{k-1} - 1)$ ，其中 k 是指数的位数。

规范化 (Normalization)

规范化使得浮点数的表示是唯一的：

- 约定**：小数点位于第一个非零有效数字的右侧，称为**规范化**。
- 二进制规范化**：尾数的最高有效位总是等于 1。
 - 形式： $\pm 0.1bbb...b \times 2^E$ 。

3. IEEE 754 标准 (IEEE 754 Standard)

IEEE 754 是表示浮点数最常见的标准，制定于 1985 年，旨在提高程序的可移植性。

单精度浮点数 (Single Precision) - 32位

字段	位数	描述
符号 (S)	1	
指数 (E)	8	范围 $0 < E < 255$ 。
尾数 (M)	23	采用 隐藏位 (Hidden Bit) 机制，实际尾数是 $1.M$ 。

- 实际指数 e : $e = E - 127$ (偏差 127)。
- 数值表示: $X = (-1)^S \times 2^{(E-127)} \times (1.M)$ 。
- 可表示范围 (幅度): 约为 1.8×10^{-38} 到 3.40×10^{38} 。
- 符合标准的要求: 计算机必须至少提供单精度表示以符合 IEEE 标准。

双精度浮点数 (Double Precision) - 64位

字段	位数	描述
符号 (S)	1	
指数 (E)	11	范围 $0 < E < 2047$ 。
尾数 (M)	52	实际尾数是 $1.M$ 。

- 实际指数 e : $e = E - 1023$ (偏差 1023)。
- 数值表示: $X = (-1)^S \times 2^{(E-1023)} \times (1.M)$ 。
- 可表示范围 (幅度): 约为 2.2×10^{-308} 到 1.8×10^{308} 。
- 双精度表示: 是可选的。

4. IEEE 754 特殊值 (Special Values)

IEEE 754 标准定义了特殊的指数和尾数组合来表示非数字和极值：

类型	符号 (S)	指数 (E)	尾数 (M)	值/描述
零 (Zero)	0/1	0	0	± 0
无穷大 (Infinity)	0/1	全 1 (255/2047)	0	$\pm \infty$ (操作溢出)
非数字 (NaN)	0/1	全 1 (255/2047)	$\neq 0$	运算结果无法确定，如 $\sqrt{-1}$
非规范化数 (Denormal)	0/1	0	$\neq 0$	非常接近 0.0，没有隐含的 1

非规范化数 (Denormal Numbers)

- 用于表示非常接近 0 的数值。
- 没有隐含的 1。
- 实际指数被假定为 $1 - \text{bias}$ 。
- 会导致**精度损失**（“渐进式下溢”）。

5. 总结与权衡 (Summary and Trade-off)

- 精度 (Accuracy) vs. 范围 (Range)**：
 - 增加**尾数**的大小，可以提高**精度**。
 - 增加**指数**的大小，可以提高可表示的**范围**。
- 扩展精度**: 提供扩展单精度 (>32位) / 扩展双精度 (>64位) 有助于减少连续计算中的**舍入误差** 并提高初等函数（如 $\sin$, $\cos$ ）的评估准确性。

好的，这是一份关于“浮点数运算 (Floating-point Number Arithmetic)”的幻灯片内容的详细总结，涵盖了浮点数加减乘除的规则以及在运算中需要考虑的关键问题。

2-9 浮点数运算总结

这份幻灯片重点介绍了在计算机中执行浮点数运算（特别是遵循 IEEE 754 标准的浮点数）所需的步骤和需要解决的精度/范围问题。

1. 浮点数加/减运算规则 (Add/Subtract Rule)

浮点数加减运算的第一步是**对齐 (Alignment)**。由于浮点数有指数部分，必须先使两个操作数的指数相等。

运算步骤（以 IEEE 单精度为例）

- 对齐 (Alignment):**
 - 选择指数较小的数，将其**尾数 (Mantissa)** 向右移位。
 - 移位步数等于两个指数的差值。
 - 同时，递增该数的指数，直到两个指数相等。
- 设置结果指数:** 结果的指数等于较大的那个指数。
- 加/减尾数:** 对尾数执行加法或减法，并确定结果的符号。
- 规范化 (Normalization):** 如果需要，对结果值进行规范化处理。

2. 浮点数乘/除运算规则

浮点数的乘除法比加减法简单，不需要尾数对齐。

乘法规则 (Multiply Rule)（以 IEEE 单精度为例）

- 指数运算:** 将两个指数相加，然后**减去偏差值 127**。
- 尾数运算:** 将两个尾数相乘，并确定结果的符号。
- 规范化:** 如果需要，对结果值进行规范化处理。

除法规则 (Divide Rule)（以 IEEE 单精度为例）

- 指数运算:** 将两个指数相减，然后**加上偏差值 127**。
- 尾数运算:** 将尾数相除，并确定结果的符号。
- 规范化:** 如果需要，对结果值进行规范化处理。

3. 浮点数运算中的问题 (Problems Considerable in FP Arithmetic)

在执行浮点数运算的中间步骤中，必须解决精度和范围限制带来的问题。

3.1 保护位 (Guard Bits)

- 目的:** 为了提高浮点计算的精度，使用**保护位**。
- 作用:** 在尾数的右端保留额外的位（保护位），并用 0 填充。
- 重要性:** 在中间步骤中保留保护位，能够最大程度地保证最终结果的准确性。

3.2 截断 (Truncation)

浮点运算可能导致尾数精度超过存储格式的限制（单精度 23 位，双精度 52 位）。在存储前，必须丢弃多余的位（截断）。

选择的截断方法应是**无偏 (unbiased)** 的，以使误差相互抵消。

截断方法	描述	偏差 (Bias)	误差范围
截断法 (Chopping)	移除保护位，不改变保留位。	有偏 (倾向于较低的尾数值)。	0 到保留位 LSB 的约 1。
冯·诺依曼取整 (Von Neumann Rounding)	如果移除位全为 0 则丢弃；如果移除位有 1，则保留位 LSB 强制置 1。	无偏。	$[-1, +1]$ LSB 位置。

截断方法	描述	偏差 (Bias)	误差范围
四舍五入法 (Rounding)	映射到最接近的可表示值。	无偏。	$[-1/2, +1/2]$ LSB 位置。

注意：四舍五入法 (Rounding) 是 IEEE 浮点数标准中指定的默认截断模式。

3.3 规范化 (Normalization)

- 在运算后，结果可能不是规范化形式。
- 非规范化的数值可以通过**移位尾数**并**调整指数**的方式，将其转换为规范化形式。

3.4 溢出和下溢 (Overflow and Underflow)

- 指数上溢 (Exponent Overflow):** 正指数超过最大可能值（如 IEEE 单精度中 $e > 127$ ）。结果可能被指定为 $\pm\infty$ 。
- 指数下溢 (Exponent Underflow):** 负指数小于最小可能值（如 IEEE 单精度中 $e < -126$ ）。数值太小而无法表示，可能被报告为 0。
- 尾数上溢 (Mantissa Overflow):** 同符号尾数相加导致最高有效位产生进位。解决方法是将结果尾数**右移一位**，并**递增指数**。
- 尾数下溢 (Mantissa Underflow):** 尾数对齐过程中，数字流失出尾数的右端。通过使用保护位和截断方法可以解决。

4. 浮点运算的实现 (Implementation)

浮点运算可以通过**软件例程**或**硬件例程**来实现。现代计算机通常提供用于浮点运算的机器指令。

您是否想看一下某个浮点数运算步骤（例如对齐或规范化）的详细示例？